Algebraic Reasoning Distiller: Harnessing multi-agent collaboration for enhanced mathematical problem-solving

Joaquín Mir Macías
joaquinmirmacias@icai.alu.comillas.es

Jose Andrés Ridruejo Tuñón
202111246@alu.comillas.edu

Javier Rios Montes
202111258@alu.comillas.edu

Miguel Montes Lorenzo
miguelmonteslorenzo@alu.comillas.edu

Abstract

This project explores the potential of multi-agent collaboration in enhancing the mathematical problem-solving capabilities of large language models (LLMs). We propose a pipeline that integrates six different agents, each specializing in a distinct aspect of mathematical reasoning, to collaboratively solve complex problems. The agents include a Planner, a Counterexample Generator, a Symbolic Prover, a RAG Retriever, an Aggregator, and a Distiller. All this is done in order to solve the problems of the Mathematics Distillation Challenge - Equational Theories.

1 Introduction

In the last few years, large language models (**LLMs**) have positioned themselves as powerful tools for a wide range of natural language processing tasks. However, their performance in *mathematical problem-solving* has been limited by their inability to perform complex reasoning and to handle symbolic manipulation effectively. Unlike human mathematicians, who are able to explicitly take into account symbolic rules and logical deductions, LLMs often operate as parrots (though stochastic ones, we must admit), aproximating solutions based on patterns rather than understanding the underlying mathematical principles. This shows in consistent errors in even simple arithmetic, spatial reasoning and symbolic manipulation tasks. Even frontier models like GPT-5 have shown these limitations, which suggests memorized reasoning templates rather than abstract understanding.

To overcome these issues, two main approaches have been explored in the literature: **pretraining** and **fine-tuning**. Pretraining involves training a model on a large corpus of mathematical texts, allowing it to learn general mathematical concepts and patterns. However, this approach can be computationally expensive. On the other hand, given both a pretrained model and a curated dataset of mathematical problems, fine-tuning allows the model to adapt to specific mathematical tasks, improving its performance without the need for extensive computational resources. This latter approach initially implied trading off generalization for specialization, turning the model into a *narrow expert*.

Recent work, though, introduced a novel paradigm that combines the strengths of both approaches: **Supervised Fine-tuning**. First theorically proposed by Stiennon et al. [1] and further implemented with **InstructGPT** [2], this method involves using a curated dataset of outputs (examples of an LLM behaving as desired) to fine-tune a pretrained model. Given that this process just tunes the model to emulate a correct style, rathen than to solve a particular task, it does not lose its generic problem-solving capabilities. In fact, it can even enhance them by providing the model with a better understanding of how to approach problems in a structured way.

Our project builds upon this idea of supervised fine-tuning, but takes it a step further by introducing a multi-agent collaboration framework. Instead of relying on a single model to solve mathematical problems, we propose a system where multiple specialized agents work together to tackle different aspects of the problem-solving process. Each agent is designed to handle specific tasks. By leveraging the strengths of each agent and allowing them to collaborate, we aim to create a more robust and effective system for mathematical problem-solving. In particular, we will apply this framework to the **Mathematics Distillation Challenge - Equational Theories** [3], which presents a set of complex mathematical problems that require a deep understanding of equational theories.

1.1 Problem Statement

Let a **magma** be a set M equipped with a binary operation $\cdot : M \times M \to M$ (no associativity, commutativity or identity element are assumed). An **equational theory** is a set of equations that hold for all elements of the magma. The goal of the Mathematics Distillation Challenge - Equational Theories is to develop a system that can effectively solve problems related to equational theories. Specifically, given two equations E_1 and E_2, the system should determine whether E_1 implies E_2 under every interpretation of the magma (this is, if every magma satisfying E_1 also satisfies E_2). A given example of this problem is the following:

$$\begin{aligned} x * y &= z * (w * (u * u)) \\ x * (y * y) &= (z * w) * z \end{aligned} \tag{1}$$

In this case, the equation is **True**. The system must generate a cheatsheet that contains the necessary information to solve this kind of problems (which is then fed to a constrained LLM that must solve the problem based on the information in the cheatsheet). The cheatsheet should include relevant definitions, properties, and examples of equational theories, as well as any other information that can help the LLM to understand and solve the problem effectively.

The competition operates under a set of constraints that limit the computational resources available to the participants:

- The generated cheatsheet must be static and no greater than 10 KB in size.
- No tools may be used during the evaluation phase, and the LLM must rely solely on the information provided in the cheatsheet to solve the problem.
- The budget for each problem is limited to 10 minutes and the equivalent of 0.01 USD in computational resources.

These constraints motivate our approach of *offline* distillation: the computationally expensive parts of the process (counterexample search, RAG,multi-agent reasoning, etc.) are performed during the training phase. during the evaluation phase, the system only needs the use of a cheatsheet and a single call to the LLM.

2 Method

Our solution, ARD (Algebraic Reasoning Distiller), follows the paradigm *distill, then deploy*. In its *offline* setting, the multi-agent system analyzes the training dataset of problems and‘produces *evidence bundles*. These are then used to both generate the final cheatsheet and to fine-tune a pretrained LLM, which is then deployed in the *online* setting to solve the test problems based on the generated cheatsheet.

2.1 System Architecture

The multi-agent system consists of 5 sequential nodes, each responsible for a specific aspect of the problem-solving process. Below, we include a graphic to illustrate the architecture of the system, followed by a detailed description of each agent's role and functionality:

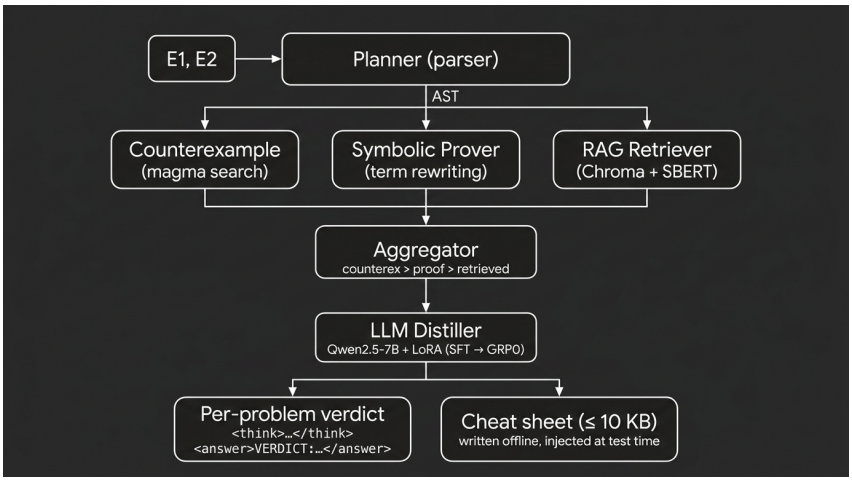

Figure 1: Architecture of the multi-agent system.

2.1.1 Planner

This agent takes the input problem ((E_1, E_2)) and generates parses them into **AST** (Abstract Syntax Tree) format. Terms are represented as nested tuples: `("var", "x")` for variables and `("op", left, right)` for operations. It implements a simple recursive descent parser to achieve this.

2.1.2 RAG Retriever

Recovers relevant information from a knowledge base of equational theories. In specific, it uses a vector database (ChromaDB) which contains the 4,694 equations from the training set, embedded using a pretrained model (`all-MiniLM-L6-v2`). The agent retrieves the *top-k* (in our case, $k = 5$) most relevant equations based on cosine similarity.

2.1.3 Symbolic Prover

This agent attempts to prove the implication $E_1 \implies E_2$ using symbolic manipulation. It applies a predefined list of strategies:

1. **Alpha-equivalence**: Check if E_1 and E_2 are the same up to variable renaming.
2. **Substitution**: Attempt to substitute terms in E_1 to derive E_2.
3. **Rewriting**: Use the retrieved equations to rewrite E_1 and see if it can be transformed into E_2.

This component is deliberately designed to be overly conservative: a negative result does not necessarily mean that the implication is false, but a positive result is a strong signal of the implication being true.

2.1.4 Counterexample Generator

Tries to find a counterexample that satisfies E_1 but not E_2. It searches increasingly complex magmas (starting from a single-element set and increasing the number of elements) and checks if there exists an interpretation that satisfies E_1 but not E_2, applying a different heuristic depending on the order:

- **Orders 2 and 3**: Exhaustive search over all possible interpretations. There are only 16 possible magmas of order 2 and 19,683 of order 3, which makes this approach feasible.
- **Order 4:** Random sampling of 20,000 Cayley tables [1] (out of the over 4 billion possible magmas of order 4) and checking if any of them satisfies E_1 but not E_2.

[1] A Cayley table is a mathematical table that describes the operation of a magma. It lists the results of applying the binary operation to every possible pair of elements in the set.

For every candidate, all possible variable assignments are checked by means of a cartesian product iteration with mixed-radix counting (to account for the different number of elements in each order). A single counterexample is sufficient to conclude that the implication is false, but if no counterexample is found, no conclusion can be drawn.

2.1.5 Aggregator

The aggregator takes the outputs of the previous agents and generates a priority policy:

- If the Symbolic Prover finds a proof, the implication is classified as **True** with confidence 0.9.
- If the Counterexample Generator finds a counterexample, the implication is classified as **False** with confidence 1.0.
- If neither of the above agents finds a conclusive result, the implication is classified as None with confidence 0.0.

An illustrative example of this pipeline can be found in Appendix A.

2.2 Architectural Details

In addition to the specific algorithms and heuristics implemented in each agent, there are two key architectural details that are worth mentioning:

- **Distilled LLM Agent**: This agent is responsible for generating the final cheatsheet based on the evidence bundles produced by the multi-agent system. It uses a pretrained LLM, in our case Qwen2.5-7B-Instruct [4], quantized to 4 bits and using BF16 precision for the attention mechanism. The model is fine-tuned on the training dataset using LoRA [5] with a rank of 16, α of 32 and a dropout of 0.05. During inference, the model operates under two different modes:
  - **Problem-specific:** Given an evidence bundle and the (optional) retrieved context, the model generates an structured answer that includes the tokens `<think>`, `</think>`, `<answer>` and `</answer>`.
  - **General-purpose:** Given a set of evidence bundles, the model generates a compact *lemma/heuristic* block using greedy decoding. This block is designed to be as informative as possible while being no greater than 1200 bytes in size (in order to fit the 10 KB limit for the final cheatsheet after including the retrieved context).
- **FastAPI Server**: A lightweight API REST exposes the functionality of the multi-agent system, allowing the access through two endpoints: POST `/sair` (which runs the complete system under a given problem, returning a `SAIRResponse` object) and GET `/health` (checking that the server is running properly).

3 Experiments and Results

All the training and evaluation processes were conducted on a node of the DGX cluster (8 $\times$ NVIDIA H100 GPUs). The training dataset consists of 4,694 problems, divided into a training set consisting of an 80% of the problems (3,755) and a validation set consisting of the remaining 20% (939).

3.1 Evaluation Metrics

The performance of our system was evaluated using the following metrics:

- **Accuracy**: Fraction of problems in which the VERDICT (True/False) is that of the ground truth.
- **Parsing Accuracy**: Fraction of outputs which contain a line with the correct format `VERDICT: <True/False>` (regardless of whether the verdict is correct or not).
- **Coverage**: Fraction of problems for which the system produces a non-empty verdict (regardless of whether it is correct or not).
- **Average Inference Time**: Average time taken to produce a verdict for a given problem.

4 Discussion

- Limitation of our approach (similar to limitations of fine-tuning in comparison to pretraining: Curation of the dataset, computational cost, etc.)

5 Conclusion

References

[1] Nisan Stiennon, Long Ouyang, Jeff Wu, Daniel Krueger, Milan Goel, Joshua Zhen, Dhruba Ganguli, Sandhini Agarwal, Danny Mislra, and Pieter Christiano. Learning to summarize from human feedback. *Advances in Neural Information Processing Systems*, 33:3008–3021, 2020.

[2] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Agarwal, Chen Wu, Danny Hadfield-Menell, Pieter Christiano, and Timothy Lyons. Training language models to follow instructions with human feedback. *arXiv preprint arXiv:2203.02155*, 2022. URL `https://arxiv.org/abs/2203.02155`.

[3] SAIR Foundation. Equational theories stage 1: Mathematics distillation challenge. `https://github.com/sair-foundation/equational-theories-stage1-judge`, 2026. Accessed: 2026-04-19.

[4] Qwen Team. Qwen2.5-7b-instruct model card. `https://huggingface.co/Qwen/Qwen2.5-7B`, 2024. Accessed: 2026-04-19.

[5] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021. URL `https://arxiv.org/abs/2106.09685`.

A Pipeline Example